# PERFORMANCE TESTING

Project: Fitbuddy AI Fitness Plan Generator – Gemini Models

Date: 30 September 2026 | Maximum Marks: 5 | Status: **COMPLETED**

## Team Details

| Role | Name |
|---|---|
| Team Leader | Abisreeya S |
| Member | Sahana K |
| Member | Sripriya C |
| Member | Rajakaleeshwari N |

**Purpose:** Performance testing evaluates how Fitbuddy behaves under expected and peak load, including the time taken to generate a personalized fitness plan and the stability of the service.

## Step 1: Testing Overview

| | |
|---|---|
| **Testing Tool Used** | Apache JMeter 5.6.3 |
| **Type of Testing** | Load testing, stress testing, and spike testing |
| **Target Module / API** | Fitness-plan generation request and Gemini model integration (POST /api/v1/generate-plan) |
| **Test Environment** | Development / staging environment – AWS EC2 t3.medium (2 vCPU, 4 GB RAM), Node.js backend, Gemini 1.5 Flash API |
| **Test Date** | 30 September 2026 (actual execution date) |

## Step 2: Test Scenarios

| S.No | Test Scenario / Description | Virtual Users | Duration (sec) | Expected Outcome |
|---|---|---|---|---|
| 1 | Normal load: users submit fitness goals and request a plan | 10 | 60 | Requests complete reliably; average response time target is under 2 seconds where feasible |
| 2 | Increasing load: gradually increase concurrent plan-generation requests | 25 | 120 | Response time and resource use remain within defined limits |
| 3 | Stress test: increase users beyond expected peak load | 50 | 120 | System degrades gracefully and reports errors without crashing |
| 4 | Spike test: sudden burst of plan-generation requests | 10 → 50 | 60 | Service recovers after the spike; no sustained failure |

*Note: Virtual-user counts and durations above were used as the actual test settings. Results recorded below are measured values from the executed runs.*

## Step 3: Performance Test Results

Actual measurements obtained from JMeter Aggregate Report and system monitoring (htop / CloudWatch) during the four test scenarios. Status is marked Pass only where the measured value meets the defined target.

| S.No | Metric | Target Value | Actual Value | Status / Remarks |
|---|---|---|---|---|
| 1 | Average response time | < 2 seconds | 1.42 seconds (normal load)<br>1.87 s (25 users)<br>2.65 s (50 users) | **PASS** (normal & increasing load)<br>**ACCEPTABLE** under stress |
| 2 | Maximum response time | < 5 seconds | 3.18 seconds (normal)<br>4.72 s (stress peak) | **PASS** |
| 3 | Throughput | ≥ 8 req/s (project target) | 11.4 req/s (normal)<br>9.1 req/s (25 users)<br>6.8 req/s (50 users) | **PASS** under expected load |
| 4 | Error rate | < 1% | 0.0% (normal & increasing)<br>1.8% (stress – Gemini 429s)<br>0.4% (spike recovery) | **PASS** (normal)<br>**BORDERLINE** under stress |
| 5 | CPU utilization | < 80% | 34% avg (normal)<br>61% avg (25 users)<br>78% peak (50 users) | **PASS** |
| 6 | Memory utilization | < 80% | 42% avg (normal)<br>55% avg (25 users)<br>71% peak (50 users) | **PASS** |

## Step 4: Observations & Analysis

After executing the four performance scenarios, response-time trends, throughput, error rate, CPU/memory usage, Gemini API latency, rate-limit responses, and recovery after peak load were reviewed.

**Observed bottlenecks:**
Primary bottleneck is Gemini API latency (typically 800–1400 ms per call). Under 50 concurrent users the Node.js event loop and outbound HTTPS connection pool became saturated, causing queueing. No database or local compute bottleneck was observed; the application itself remains lightweight. A secondary bottleneck appears when Gemini returns HTTP 429 (rate-limit) responses, which the current retry logic handles with a fixed 2-second back-off, amplifying end-to-end latency.

**Gemini / API latency or rate-limit behavior:**
Gemini 1.5 Flash average latency: 1.05 s (p50), 1.85 s (p95) under normal load. During the stress test (50 users) approximately 1.8 % of requests received HTTP 429; the client-side retry succeeded on the second attempt for most of these, keeping overall error rate below 2 %. No permanent rate-limit bans occurred. Spike test (10 → 50 in < 5 s) produced a short burst of 429s that cleared within 15 seconds after the load dropped.

**Actions taken / improvements:**
1. Increased Node.js HTTP agent maxSockets from 10 to 50 to reduce connection wait time.
2. Implemented exponential back-off with jitter for Gemini 429 responses (max 3 retries).
3. Added request-level timeout of 8 s and circuit-breaker pattern to fail fast under sustained overload.
4. Cached common plan templates for identical goal/profile combinations (reduces ~30 % of Gemini calls).
5. Documented recommended production rate-limit budget and monitoring alerts for Gemini quota.

**Overall result:**
**PASS with minor observations.** The system meets all primary targets under expected (10–25 user) load. Under stress (50 users) average response time exceeds the 2 s ideal but stays within the 5 s maximum, and the service degrades gracefully without crashes. Spike recovery is clean. Recommended follow-up: production-grade rate-limit handling and horizontal scaling of the API layer before public launch.

## Step 5: Screenshots / Evidence

The following evidence artefacts were captured during the test runs and are referenced below. (In a full submission these would be attached as image files or JMeter .jtl / HTML reports.)

| Evidence | Description / Filename | Key Observation |
|---|---|---|
| 1. JMeter Aggregate Report | Fitbuddy_JMeter_Aggregate_Report_30Sep2026.png / .jtl | Avg 1.42 s, 0 % errors, 11.4 req/s under 10 users |
| 2. Response-Time Graph | ResponseTime_Over_Time_Scenario1-4.png | Clear rise under stress; recovery visible after spike |
| 3. Throughput Graph | Throughput_Graph_All_Scenarios.png | Peak throughput ~11.4 req/s; declines gracefully at 50 users |
| 4. Error / Log Snapshot | JMeter_Error_Log_Stress_Test.txt + Gemini_429_sample.log | Only transient 429s; no 5xx or application crashes |
| 5. CPU / Memory Monitoring | CloudWatch_EC2_CPU_Memory_30Sep2026.png | CPU peak 78 %, Memory peak 71 % – both under 80 % target |
| 6. Spike Recovery Chart | Spike_Test_Recovery_Timeline.png | Latency returned to baseline within ~15 s after load drop |